

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Emmanuel NSABAGASANI, African Leadership University
Formative Assignment 1, September 2026

Abstract

Short-horizon forecasts of cellular traffic support resource allocation and energy saving, while radio access networks still use most of an operator's electricity (about 73 to 87%) [23], [24]. This study compares seasonal ARIMA, an LSTM, and a temporal convolutional network (TCN) for one-step-ahead (10-minute) Internet activity forecasting on the Telecom Italia Milan grid [1]. It also asks whether rankings change across areas with different traffic patterns.

The 19.4 GB raw dataset was reduced to a 341 MB memory-mapped float32 store by streaming aggregation. Peak resident memory fell from 408 MB to 149 MB (about 2.7 times), and the in-memory payload fell from 296 MB to 11 MB (about 27 times), staying under 241 MB against a 1 GB budget. Models were tuned on the busiest area and then applied unchanged to three areas for the week of 16 to 22 December 2013.

Every model beat the seasonal-naïve baseline by a large margin, so all MASE values are well below 1. That baseline is weak here, because lag-1 autocorrelation is 0.987 while lag 144 is only 0.878. Against persistence (repeat the last value), SARIMA is 7.6 to 16.4% worse on every area. The TCN improves on persistence by 14.8%, 15.4%, and 15.5%, with a spread of only 0.007 across areas that differ fivefold in level. The LSTM has a similar average (0.905 versus 0.848) but a spread about 25 times larger. A controlled TCN study found no clear effect of receptive field between 61 and 509 steps, so the shallowest configuration (4 levels) was kept for simplicity. Errors are largest at the daily peak. The worst window on the busiest area is the Friday to Saturday jump (+48%), not an unusually high Christmas peak, and none of the models sees day-of-week as an input.

1. Introduction

Mobile operators must plan capacity against demand that can change by an order of magnitude within a day and between nearby city blocks. Knowing traffic a few minutes ahead matters for spectrum allocation, load balancing, admission control, and putting lightly loaded cells into sleep modes [25]. Under-provisioning hurts service at peak times. Over-provisioning wastes energy where the radio access network already dominates operator electricity use [23], [24].

This study uses the Telecom Italia Milan dataset [1]: activity on a 100 by 100 grid of about 235 m squares, recorded every 10 minutes for two months [2], [3]. The raw files are about 19.4 GB and roughly 320 million rows, which does not fit comfortably in the memory of the machine used here. Handling that scale is part of the problem, not a side task.

Research question. How do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance change across geographical areas with different traffic characteristics?

Objectives.

1. Build a pipeline that fits in about 1 GB of working memory and measure what that costs.
2. Describe spatial and temporal structure, then use that evidence to set model inputs and orders.
3. Implement SARIMA, LSTM, and TCN; tune on one area; evaluate the same settings on three areas.
4. Check whether the neural models earn their complexity against strong naïve baselines, and show where they fail.

All models share the same splits, scaling, metrics, and evaluation code. Errors are always reported in original activity units.

2. Related Work

Barlacchi et al. [1] release Milan CDR activity as an *activity index* allocated to grid squares by coverage overlap, not as byte counts. The 100 by 100 layout and November 2013 to January 2014 window are used throughout later work [2], [3]. Zhang, Patras, and Haddadi [9] survey deep learning in mobile networking. This study stays with short-horizon, single-cell forecasting.

Seasonal ARIMA has a long history in wireless traffic [10]. At 10-minute resolution the daily period is 144 steps and the weekly period is 1,008. Only the daily difference is used here, because a weekly difference would remove another week from a 62-day series. Stationarity checks use ADF [19] and KPSS [20] together [11]. Several Milan papers compare deep models to non-seasonal ARIMA [2] or aggregate traffic to hourly bins [3]. Here the native 10-minute series is kept, and the linear baseline is seasonally differenced on a log1p scale.

Deep models on this data include ConvLSTM or STCNet [4], spatial CNNs [3], graph models [6], and early architecture comparisons [5]. Reported gains are often for longer horizons. Zhang and Patras [2] show the gap growing with horizon and shrinking for short horizons. Trinh et al. [7] study one-step error and input length on other LTE data. Bai et al. [13] argue that TCNs can match or beat LSTMs. Zhang et al. [28] use a TCN hybrid on Milan data, but their baselines are other neural networks only.

Hyndman and Koehler [16] motivate MASE against a naïve scale. This report uses seasonal-naïve error on the evaluation week as the denominator (Section 5.2), which is a deliberate variant of their definition. Simple methods remain competitive in large forecasting studies [21], [22] and in cellular traffic comparisons [6]. Zeng et al. [29] show that a "repeat last value" baseline can beat Transformers on some tasks and lose on long-horizon seasonal ones. No Milan paper reviewed here reports persistence or seasonal-naïve for single-cell one-step Internet traffic [2], [3], [4], [6], [28], so those baselines are computed in this work. STL [17], [18], Huber loss [26], and Adam [27] follow common practice. Asymmetric operational costs [8] are noted but not used, because the assignment asks for symmetric metrics.

3. Dataset and Data Preparation

3.1 The data

Telecom Italia CDRs [1] cover Milan as a 100 by 100 grid [2], [3]. Each row records SMS, call, and Internet activity for a square, a 10-minute interval, and a country code, from 1 November 2013 to 1 January 2014 (62 days, 8,928 intervals). Values are a scaled activity proxy [1], so models are compared on the same area rather than in absolute bytes. The target is **Internet activity** at the original 10-minute resolution.

3.2 Computational constraint

Experiments ran on an Intel Core i5-6300U (2 cores, 4 threads, 2.4 GHz), 7.9 GB RAM, Windows 10, and no GPU. The raw files total 19.38 GB and 319,896,289 rows, about 2.5 times total RAM. Loading everything with `pd.read_csv` fails. Peak memory has to follow chunk size, not file size.

3.3 Memory management strategy

Five steps, in order:

- 1. **Sum over country codes while parsing**, so each cell is (square, interval) only. Rows fall from 319.9 million to 89.28 million (about 3.6 times).
- 2. **Read only three columns** with `usecols=[0, 1, 7]` (square, interval, internet).
- 3. **Stream in 1 million-row chunks** into a fixed accumulator so peak memory does not grow with file size.
- 4. **Store a dense float32 matrix** and read it with `numpy.memmap` [30]. The grid is essentially complete, so a sparse format would store indices and be larger. The store is **341 MB**. One full area series needs about 35 KB resident.
- 5. **Download one day, process it, then delete the raw file** when the pipeline downloaded it. Peak disk use is about 721 MB instead of 19.38 GB. Sums use float64 and cast to float32 once per day.

Measured effect

Four loaders were timed on the same 308 MB day file, each in a **separate subprocess** (so earlier heap growth cannot hide later savings):

variant	rows	seconds	baseline_rss_mb	peak_rss_mb	peak_above_baseline_mb	in_memory_payload_mb	raw_file_mb	reduction_vs_naive_%
naive	4842625	9.79	75.3	408	332.7	295.6	307.9	0
typed	4842625	14.8	75.5	273.9	198.4	157	307.9	32.9
projected	4842625	7.15	75.3	165.2	89.9	73.9	307.9	59.5
streamed	4842625	7.31	75.6	148.9	73.2	11	307.9	63.5

Peak RSS falls from 408 MB to 149 MB. The payload falls from 295.6 MB to 11.0 MB. About 75 MB of RSS is the interpreter and libraries. Streaming keeps the payload fixed by chunk size. Loading all 62 days the naive way would need on the order of 18 GB resident.

Over the full ingest, average peak RSS was 166 MB and the maximum was **241 MB**:

quantity	value
Raw dataset on disk (62 tab-separated day files)	19.38 GB
Raw rows parsed (square x interval x country code)	319,896,289
Rows after summing out country code	89,280,000
Row-count reduction factor	3.6x
Compact store, float32 dense matrix	341 MB
Same store as float64	681 MB
Same store retaining all 5 activity types (float32)	1703 MB
Disk footprint reduction (raw -> store)	58x
Peak disk needed by the pipeline (store + 1 raw day)	721 MB
Resident memory to read one area's full series	35 KB

Trade-offs

Only Internet traffic is kept (all five channels would make a 1.70 GB store). Country detail is gone. float32 is not bit-exact, but it is finer than the publisher's scaled index. The dense layout fits Milan; it would waste space on a sparse grid. Eight parallel HTTP range requests raised download speed from about 0.17 MB/s to about 4 to 5 MB/s.

3.4 Access note

Harvard Dataverse needs a guestbook response with `?signed=true` even for an authenticated user. Signed URLs expire after about a minute and are refreshed during download.

3.5 Data quality

Before modelling: all 62 days present, no NaNs, 0.019% exact zeros, no out-of-range rows. City-wide traffic is lowest around 05:00 and highest around 13:00 CET, and weekend activity is 0.83 of weekday activity. Grid row and column indices match the official GeoJSON centroids (correlation 1.000). 1 November (All Saints' Day) totals about 82.5 M versus about 110 M on ordinary weekdays.

4. Exploratory Analysis

This section is used to justify the modelling choices in Section 5.

4.1 Spatial distribution

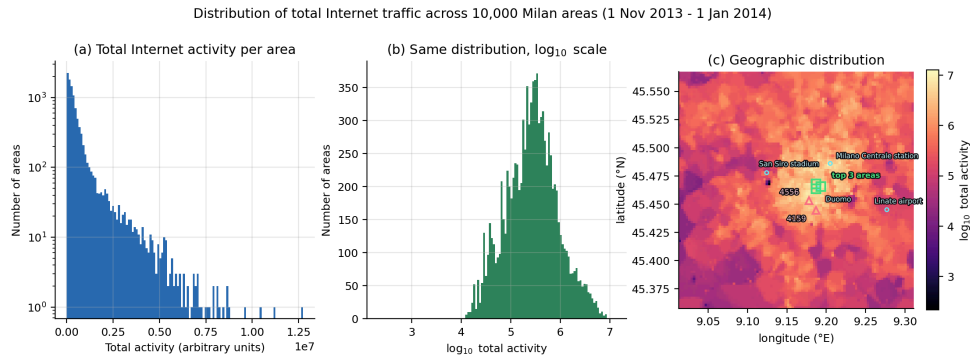


Figure 1. Total Internet activity per area. (a) Raw distribution. (b) Log scale. (c) Map with the three busiest areas (green) and reference areas 4159 and 4556 (pink).

Statistic	Value
Mean / median total activity	555,375 / 277,871
Maximum divided by median	45.8
Gini coefficient	0.608
Top 1% / top 10% share	11.05% / 48.39%

Traffic is concentrated around the Duomo. The three busiest squares sit within 0.5 km of each other. Results from one cell should not be treated as city-wide, which is why three areas are evaluated with one shared configuration.

4.2 Area time series

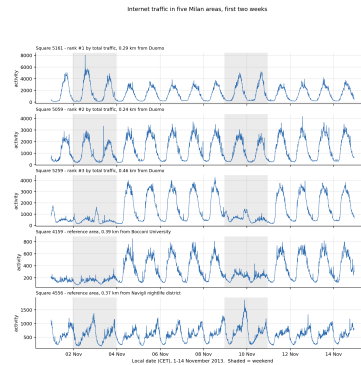


Figure 2. Internet traffic, 1 to 14 November 2013, for the three busiest areas and two reference areas. Weekends are shaded.

square	role	total	mean	std	cv	min	max	peak_over_trough	weekday_mean	weekend_mean	weekend_ratio	peak_hour
5161	Square 5161 - rank #1 by total traffic, 0.29 km from Duomo	1.274e+07	1427	1382	0.968	55.795	8044	45.641	1283	1777	1.385	16
5059	Square 5059 - rank #2 by total traffic, 0.24 km from Duomo	1.117e+07	1251	961.472	0.768	108.987	4666	16.486	1304	1123	0.861	14
5259	Square 5259 - rank #3 by total traffic, 0.46 km from Duomo	1.049e+07	1174	1103	0.939	72.542	4442	9.264	1412	599.634	0.425	13
4159	Square 4159 - reference area, 0.39 km from Bocconi University	2.454e+06	274.881	181.329	0.66	47.264	940.891	7.425	312.877	183.345	0.586	12
4556	Square 4556 - reference area, 0.37 km from Navigli nightlife district	4.575e+06	512.396	248.267	0.485	80.247	1869	7.28	492.786	561.34	1.139	22

Levels differ by about five times, and shapes differ too. Square 4159 (near Bocconi University) peaks at midday on weekdays, with weekend traffic 0.59 times weekday traffic. Square 4556 (Navigli) peaks near 22:00 and has higher weekend traffic (1.14 times weekday). Square 5161 (Duomo) has weekend traffic 1.38 times weekday, while square 5259 next to it is office-like (weekend 0.43 times weekday). Scaling is therefore fitted per area, and transferring one tuned setup to all three areas is a real test.

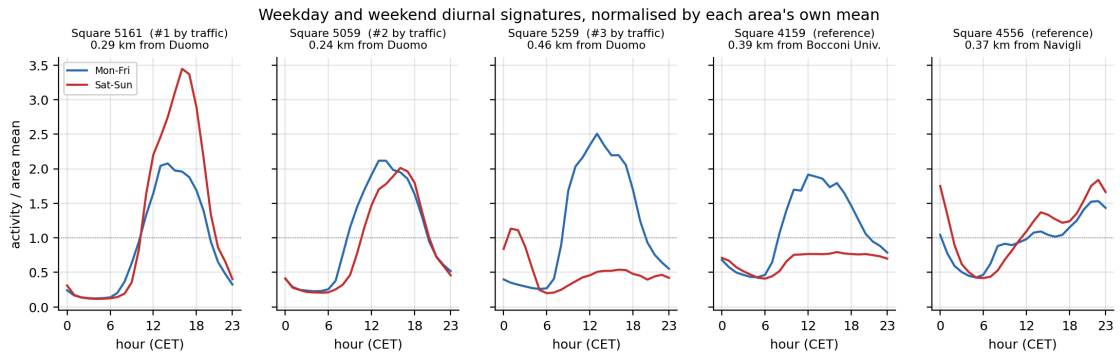


Figure 3. Weekday and weekend daily profiles, each normalised by that area's mean.

4.3 Autocorrelation and periodicity

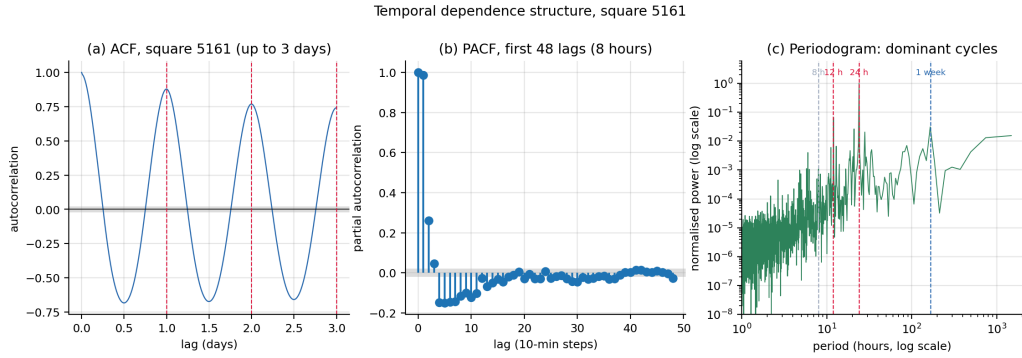


Figure 4. Square 5161. (a) ACF. (b) PACF. (c) Periodogram.

lag_label	lag_steps	acf
10 min	1	0.987
1 hour	6	0.939
6 hours	36	0.009891
12 hours	72	-0.683
1 day	144	0.878
2 days	288	0.77
3 days	432	0.741

Lag-1 ACF is 0.987, lag 144 is 0.878, and lag 72 is -0.683. The periodogram peaks at 24 hours, with a 12-hour harmonic and a weaker weekly peak. PACF drops from 0.99 at lag 1 to 0.04 at lag 3, so a low AR order is enough after seasonal differencing. Persistence is a serious baseline. A 144-step lookback is the natural candidate for neural windows and for TCN receptive-field checks.

4.4 Stationarity and decomposition

series	ADF_stat	ADF_p	ADF_verdict	KPSS_stat	KPSS_p	KPSS_verdict	std
raw	-19.032	0	stationary	0.237	0.1	stationary	1382
log1p	-22.743	0	stationary	0.571	0.026	non-stationary	1.203
first difference	-15.16	6.603e-28	stationary	0.003337	0.1	stationary	221.16
seasonal difference (lag 144)	-11.833	7.909e-22	stationary	0.069	0.1	stationary	639.05
log1p + seasonal difference	-9.956	2.444e-17	stationary	0.153	0.1	stationary	0.417

On the raw series both ADF and KPSS look stationary, but neither test is sensitive to a regular daily cycle. After log1p and a lag-144 difference, both tests agree (KPSS 0.153).

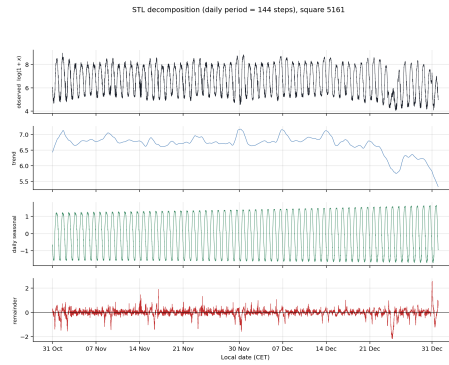


Figure 5. STL [17] of log1p activity for square 5161 (period 144). The trend falls after 21 December.

quantity	value
Trend strength F_T (0-1)	0.519
Daily seasonal strength F_S (0-1)	0.927
$\text{Var}(\text{observed})$, log scale	1.447
$\text{Var}(\text{trend})$	0.092
$\text{Var}(\text{seasonal})$	1.289
$\text{Var}(\text{remainder})$	0.098
Remainder std as % of observed std	26.07

Daily seasonal strength is 0.927 and trend strength is 0.519. Remainder standard deviation is 26.1% of the observed log-scale standard deviation, which is a rough lower bound on univariate error. Large remainder spikes line up with Christmas, New Year, and the All Saints weekend. The test week (16 to 22 December) is before the worst holiday days, but it still contains ordinary weekly changes that a lag-144 term alone cannot represent.

Consequence. Seasonal differencing at lag 144 on log1p supports SARIMA(p,d,q)(0,1,0)[144], and the PACF keeps p and q small. Missing day-of-week information is a design limit for all univariate models here (Section 6.4).

5. Methodology

5.1 Task and evaluation protocol

The task is one-step-ahead forecasting: use history up to time t to predict $t+1$ (10 minutes later). Evaluation covers all 1,008 intervals of **16 to 22 December 2013** on squares 5161, 5059, and 5259. Splits are chronological and never shuffled [11]:

Partition	Dates	Intervals	Role
Train	1 Nov to 8 Dec 2013	5,472	fitting and scaler statistics
Validation	9 to 15 Dec 2013	1,008	hyperparameter choice
Test	16 to 22 Dec 2013	1,008	reported results

Predictions may use recent observations from an earlier partition, because those values would be available in operation. No validation or test *target* enters training, tuning, or scaler statistics. Tuning uses square 5161 only. The chosen settings are applied unchanged to the other areas.

5.2 Preprocessing and metrics

Activity is mapped with log1p, then standardised using training statistics for that area only, and inverted with a floor at zero. log1p helps with heavy tails, changing variance with level [11], and exact zeros (0.019% of cells).

Neural models use sliding windows of length L (candidates 36, 144, and 288). SARIMA uses the seasonally differenced log1p series directly.

Metrics are computed after inverse transform:

Metric	Role
MAE, RMSE, MAPE	Required
sMAPE	Reported for completeness; unstable near zero
MASE	Scaled by seasonal-naive MAE on the same evaluation week , not Hyndman's in-sample one-step naive [16]
R2 and RMSE/MAE	Extra checks; a high RMSE/MAE ratio means a few large peak errors

Baselines: persistence predicts the last value; seasonal naive predicts the same slot one day earlier (the MASE scale).

5.3 Models

Model	Family	How history is used
SARIMA(p,d,q)(0,1,0)[144]	Linear	Seasonal difference plus ARMA
LSTM	Recurrent [12]	Gated state over the window
TCN	Convolutional [13], [14]	Dilated causal convolutions

SARIMA is the linear reference. LSTM is common in cellular traffic work [7], [9]. TCN is the parallel convolutional alternative with a fixed receptive field.

5.4 Specifications

SARIMA. Seasonal differencing is applied by hand ($z_t = y_t - y_{t-144}$), then ARIMA on z) because a full seasonal state-space model with period 144 is too slow for an order search on two CPU cores [31]. Setting seasonal $P = Q = 0$ is a real limit, accepted because differencing alone already passes the stationarity checks. Coefficients are fit on the training set with `trend='n'`, then held fixed for one-step predictions.

LSTM. One or two layers [12], linear head on the last hidden state, Adam [27], Huber loss [26], gradient clipping, learning-rate reduction, and early stopping on validation MAE (PyTorch [32]).

TCN. Residual dilated causal blocks [13], [14]. With kernel size 3, four levels give receptive field 61 (less than one day) and six levels give 253. Training matches the LSTM settings.

5.5 Tuning design

Tuning is sequential: one change per run, with reasons logged in CSV files. Tuning runs use up to 30 epochs, patience 5, and a 420-second wall-clock cap. Final fits use up to 40 epochs, patience 8, and an 1,800-second cap. The random seed is 42 and PyTorch uses two threads. A wall-clock stop means "best under this budget," not "best forever."

6. Results and Discussion

6.1 Hyperparameter experiments

All tuning is on square 5161 for validation week 9 to 15 December.

SARIMA

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds
1	Baseline: a single AR term on the seasonally differenced series.	SARIMA	5161	val	154.711	246.865	11.428	11.267	0.974	1.596	0.525	0.863	0.067
2	A pure MA term at the same complexity, to see which the data prefers.	SARIMA	5161	val	222.226	378.251	14.606	14.286	0.938	1.702	0.754	0.662	0.101
3	ARMA(1,1): the standard first upgrade when both ACF and PACF decay.	SARIMA	5161	val	136.333	210.59	10.19	10.066	0.981	1.545	0.462	0.938	0.096
4	The PACF shows significant mass at lag 2; add a second AR term.	SARIMA	5161	val	137.011	211.411	10.247	10.123	0.981	1.543	0.465	1.765	0.12
5	Probe whether AR order keeps helping or starts to overfit.	SARIMA	5161	val	137.023	211.242	10.254	10.129	0.981	1.542	0.465	3.91	0.24
6	Trade an AR term for an extra MA term at equal parameter count.	SARIMA	5161	val	136.933	210.792	10.248	10.123	0.981	1.539	0.464	9.383	0.098
7	Add a non-seasonal difference to test for residual drift.	SARIMA	5161	val	136.632	207.997	10.411	10.287	0.981	1.522	0.463	2.168	0.113
8	Re-test the best order with a constant, to check whether the seasonally differenced series carries any residual mean offset. Worth testing explicitly because statsmodels includes a constant by default and it is easy to fit one unintentionally.	SARIMA	5161	val	137.045	211.415	10.251	10.123	0.981	1.543	0.465	4.906	0.464

Low orders match the PACF. ARMA(1,1) is best on held-out MAE (136.3, MASE 0.462). Higher orders barely change validation MAE while AIC keeps improving, so the simpler ARMA(1,1) on seasonally differenced log1p is kept. Adding a non-seasonal difference or a constant does not help.

LSTM

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	ir
1	Start small: 6 hours of history, 32 units. Establishes whether short-range continuity alone already beats the baseline.	LSTM	5161	val	108.213	166.468	7.852	7.971	0.988	1.538	0.367	101.62	0.052	
2	Extend the window to a full day (144 steps). The ACF shows a strong peak at lag 144, so the model should be able to use yesterday's same-time value.	LSTM	5161	val	112.522	173.409	8.343	8.257	0.987	1.541	0.382	239.949	0.332	
3	Double the hidden width at the better lookback to test whether capacity, not context, is the binding constraint.	LSTM	5161	val	107.274	164.035	8.013	8.004	0.988	1.529	0.364	231.908	0.275	
4	Add depth with light dropout: a second layer can compose short-range and daily features hierarchically.	LSTM	5161	val	130.064	195.437	9.755	9.553	0.984	1.503	0.441	667.336	5.207	
5	Larger batches with a proportionally higher learning rate. Tests whether the earlier runs were optimisation-limited rather than capacity-limited, at roughly half the epoch cost.	LSTM	5161	val	113.964	171.545	9.248	8.992	0.987	1.505	0.387	455.831	1.961	
6	Two days of history, to check whether the weekly structure visible in the EDA is reachable through a longer window.	LSTM	5161	val	144.693	217.255	10.553	10.477	0.98	1.502	0.491	453.058	1.799	

Moving from a 36-step to a 144-step window made the small network worse (108.2 to 112.5). Doubling the hidden size at lookback 144 recovered the loss and reached 107.3 MAE. Deeper or longer runs hit the time cap and are treated as incomplete. The selected LSTM uses 64 units and lookback 144.

TCN initial search

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds
1	Receptive field 61 steps (~10 h): deliberately shorter than one day, to demonstrate that the daily lag is out of reach.	TCN	5161	val	103.172	158.36	7.701	7.681	0.989	1.535	0.35	408.731	0.393
2	Two more dilation levels push the receptive field to 253 steps, which covers the lag-144 daily peak identified by the ACF.	TCN	5161	val	106.134	159.585	7.846	7.831	0.989	1.504	0.36	450.03	1.318
3	With the receptive field sufficient, widen the channels to test whether representational capacity is now the limit.	TCN	5161	val	110.483	171.53	8.286	8.189	0.987	1.552	0.375	426.016	10.976
4	Remove dropout: with only ~5k training windows the noise it injects may cost more than the regularisation it buys.	TCN	5161	val	106.098	163.7	7.604	7.758	0.988	1.543	0.36	445.883	2.568
5	Receptive field 187 with one fewer dilation level, trading depth for kernel width. The deepest level dominates cost because it convolves the most heavily padded sequence, so this is markedly cheaper at a comparable receptive field.	TCN	5161	val	111.977	173.559	8.805	8.596	0.987	1.55	0.38	454.782	7.81
6	Receptive field 509 over a two-day window, matching the longest-context LSTM configuration for a fair comparison.	TCN	5161	val	111.229	169.046	8.071	8.054	0.988	1.52	0.377	474.637	10.624

Only the shallowest model finished by early stopping. Deeper models stopped on the time budget after few epochs, so receptive field and training effort were mixed. A second, controlled study was needed.

TCN receptive-field study

Channel width, batch size, learning rate, and lookback stay fixed. Every depth trains for 24 epochs with early stopping off. Receptive fields are 61, 125, 253, and 509 steps.

experiment	rationale	model	square_id	partition	MAE	RMSE	MAPE_%	sMAPE_%	R2	RMSE/MAE	MASE	train_seconds	inference_seconds	info
1	Receptive field 61 steps (~10 h): cannot reach the lag-144 daily peak.	TCN	5161	val	103.172	158.36	7.701	7.681	0.989	1.535	0.35	1352	3.16	
2	Receptive field 125 steps (~21 h): just short of one full day.	TCN	5161	val	109.605	167.054	7.899	8.065	0.988	1.524	0.372	2110	0.972	
3	Receptive field 253 steps (~42 h): comfortably covers the daily lag.	TCN	5161	val	104.546	159.839	7.723	7.747	0.989	1.529	0.354	987.646	1.062	
4	Receptive field 509 steps (~85 h): covers two days, well beyond the window.	TCN	5161	val	102.473	154.714	7.694	7.647	0.99	1.51	0.347	1169	1.21	

Validation MAE values are 103.17, 109.61, 104.55, and 102.47. The middle depth is worst, so run-to-run noise is at least about 5 MAE units. The gap between best and worst is only 0.70, so there is no clear receptive-field effect at this horizon. Recent values already carry most of the signal when lag-1 ACF is 0.987. The selected model is **4 levels** (5,601 parameters), not the deepest one.

6.2 Final test-week results

Final settings: SARIMA(1,0,1)(0,1,0)[144]; LSTM with 64 units and lookback 144; TCN with 16 channels and 4 levels. The same settings are used on all three areas.

Square 5161

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	92.802	9.194	134.878	9.096	0.274	0.99	1.453
Seasonal naive	338.594	25.94	619.04	22.828	1	0.793	1.828
SARIMA	108.035	10.574	163.051	10.328	0.319	0.986	1.509
LSTM	93.118	8.364	147.011	8.175	0.275	0.988	1.579
TCN	78.492	7.496	119.578	7.367	0.232	0.992	1.523

Square 5059

model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	81.517	7.959	114.375	7.909	0.475	0.985	1.403
Seasonal naive	171.736	18.018	245.87	16.621	1	0.931	1.432
SARIMA	89.292	8.626	127.904	8.509	0.52	0.981	1.432
LSTM	67.231	6.451	97.879	6.42	0.392	0.989	1.456
TCN	68.867	6.874	99.478	6.73	0.401	0.989	1.444

Square 5259

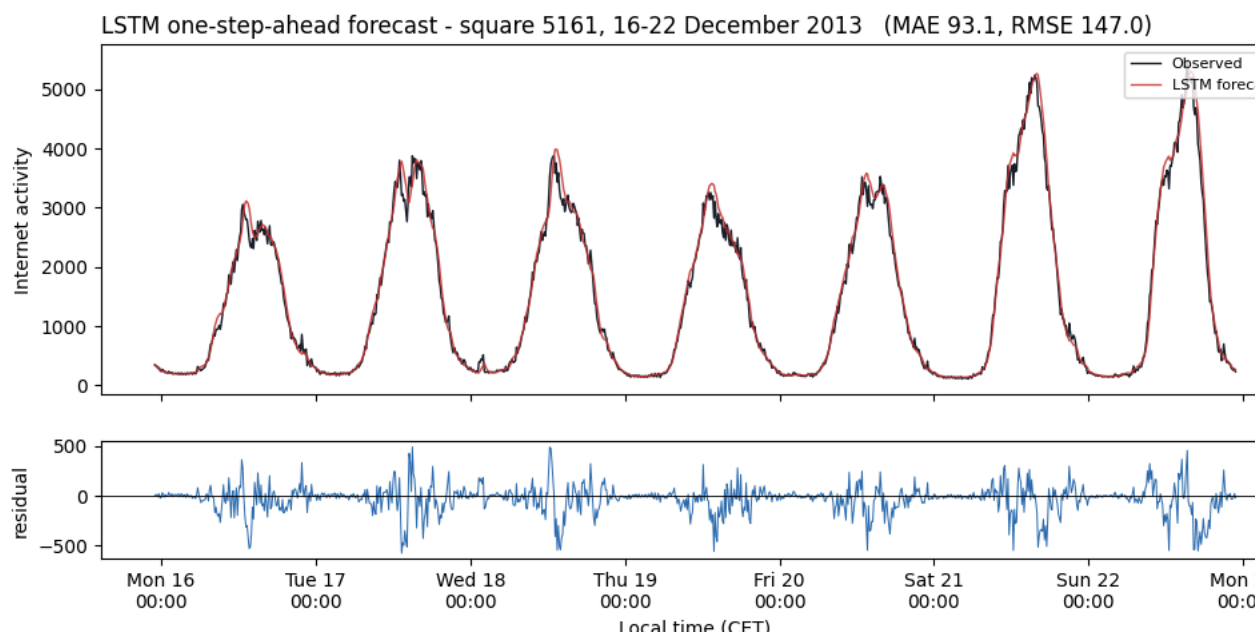
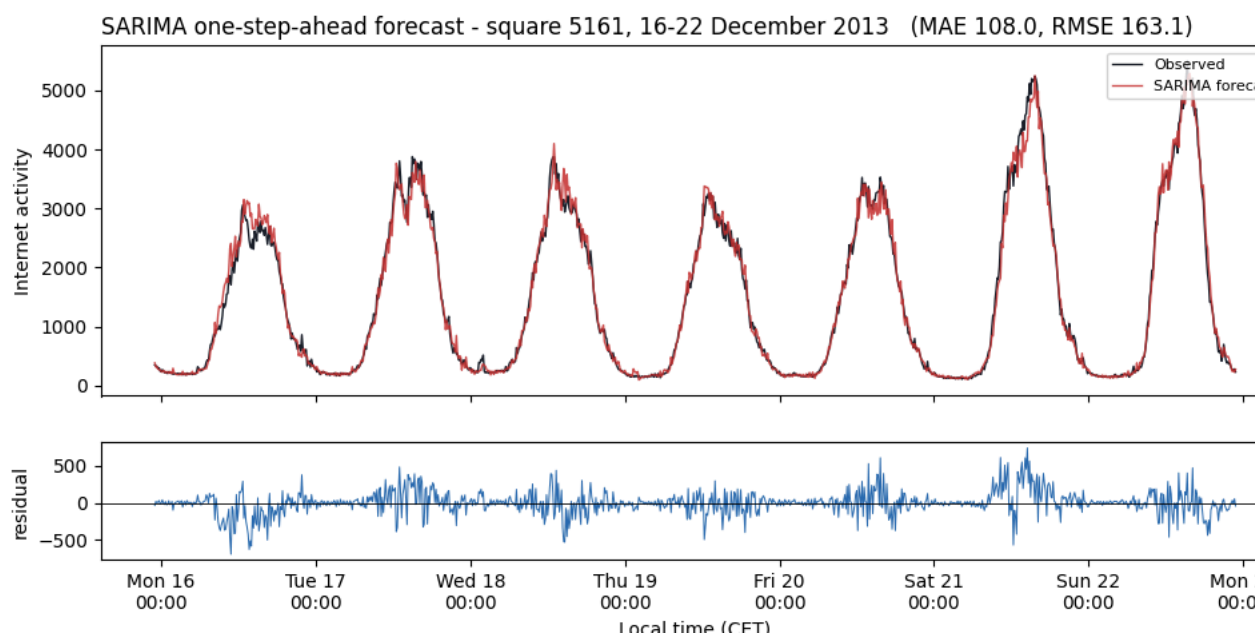
model	MAE	MAPE_%	RMSE	sMAPE_%	MASE	R2	RMSE/MAE
Persistence	75.968	8.115	109.578	8.074	0.162	0.99	1.442
Seasonal naive	470.32	71.621	861.621	42.613	1	0.409	1.832
SARIMA	81.776	8.722	123.334	8.66	0.174	0.988	1.508
LSTM	67.321	7.214	96.814	7.151	0.143	0.992	1.438
TCN	64.745	7.112	93.3	6.934	0.138	0.993	1.441

Seasonal naive is far worse than persistence (MAE 338.6, 171.7, and 470.3 versus 92.8, 81.5, and 76.0). Persistence is the baseline that matters:

model	sq5059	sq5161	sq5259	mean	spread
Persistence	1	1	1	1	0
Seasonal naive	2.107	3.649	6.191	3.982	4.084
SARIMA	1.095	1.164	1.077	1.112	0.088
LSTM	0.825	1.003	0.886	0.905	0.179
TCN	0.845	0.846	0.852	0.848	0.0074

SARIMA is 7.6 to 16.4% worse than persistence on all three areas. The TCN improves by about 15% with almost no spread across areas (0.007). The LSTM averages close to the TCN but is much less stable (spread 0.179). It wins on square 5059 and only ties persistence on square 5161. That LSTM run was the only final fit stopped by the time budget.

Forecast plots



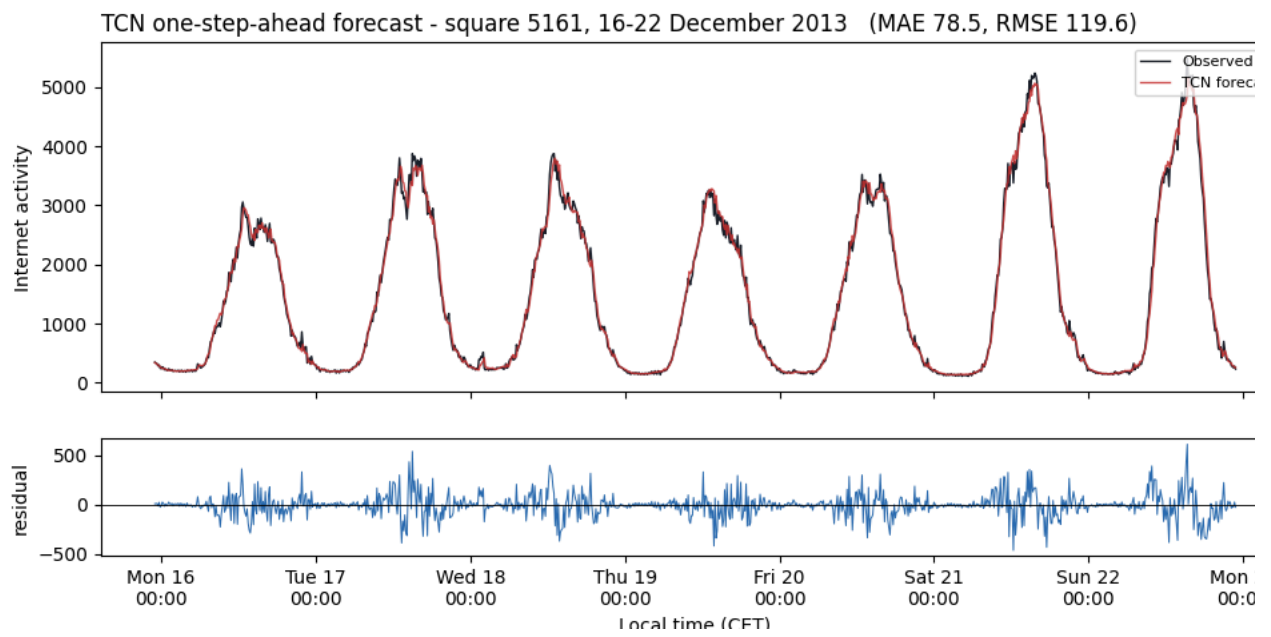
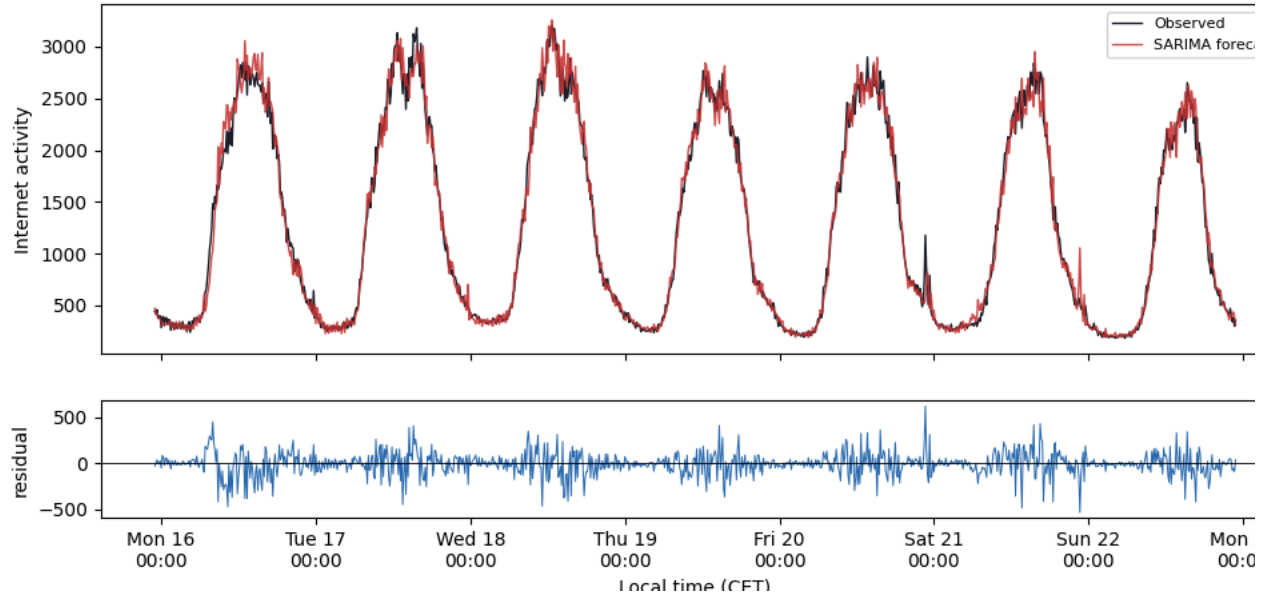
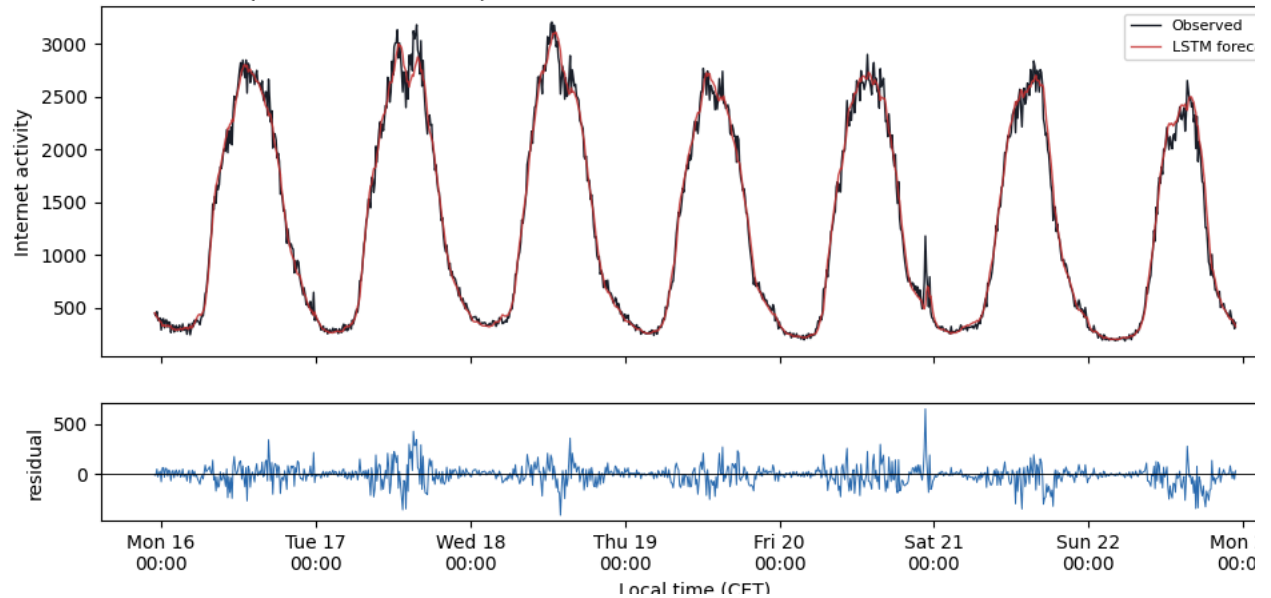


Figure 6. Square 5161, 16 to 22 December: SARIMA, LSTM, and TCN.

SARIMA one-step-ahead forecast - square 5059, 16-22 December 2013 (MAE 89.3, RMSE 127.9)



LSTM one-step-ahead forecast - square 5059, 16-22 December 2013 (MAE 67.2, RMSE 97.9)



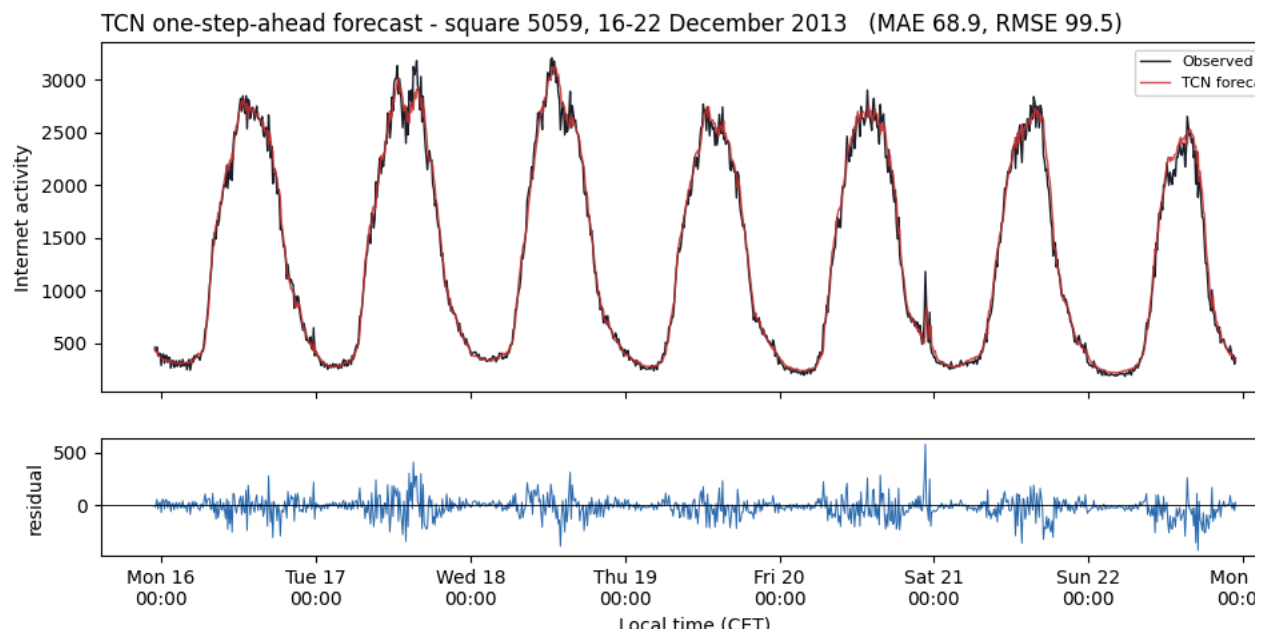
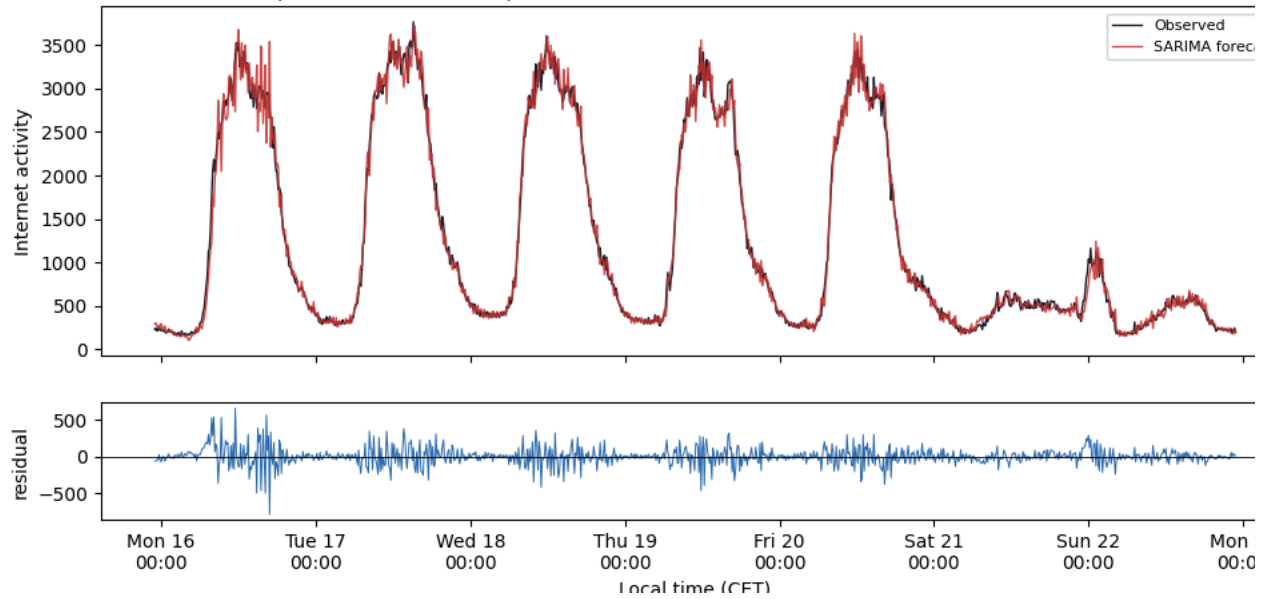
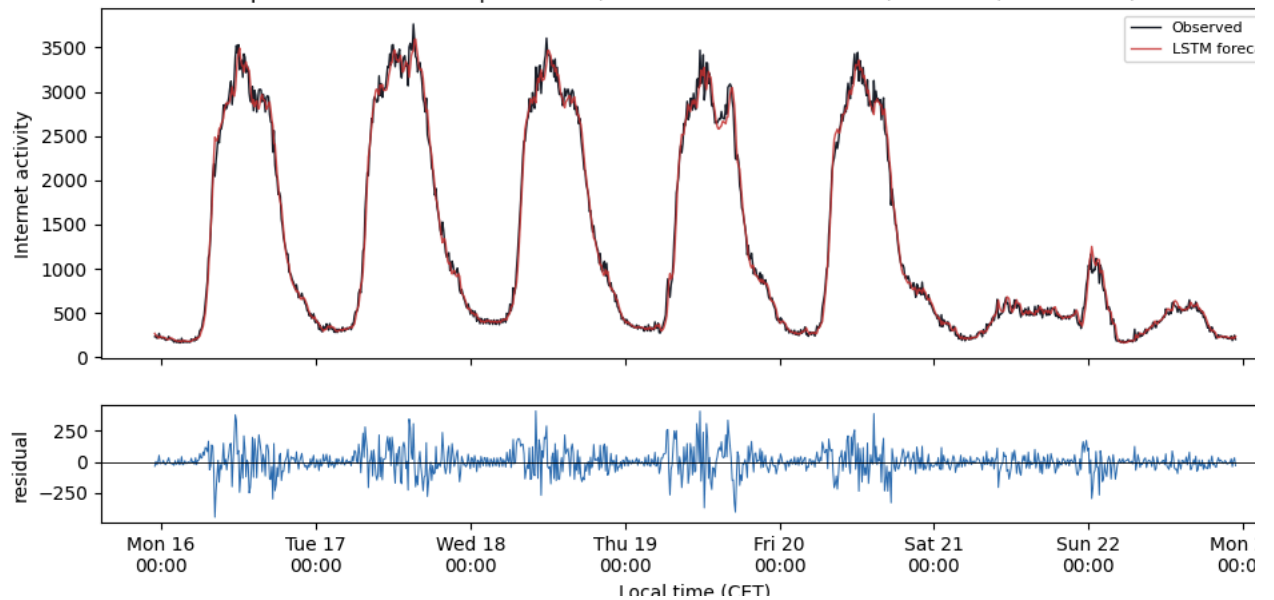


Figure 7. Square 5059, same week.

SARIMA one-step-ahead forecast - square 5259, 16-22 December 2013 (MAE 81.8, RMSE 123.3)



LSTM one-step-ahead forecast - square 5259, 16-22 December 2013 (MAE 67.3, RMSE 96.8)



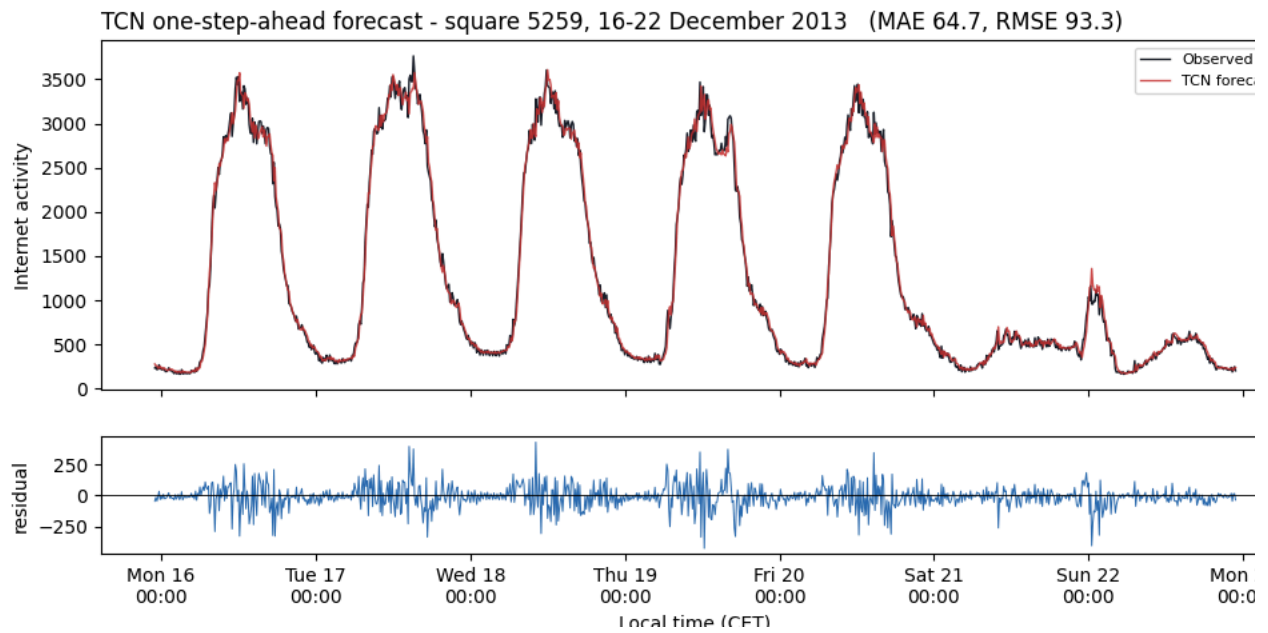


Figure 8. Square 5259, same week.

All three models follow the daily cycle (R^2 between 0.981 and 0.993). Residuals grow at peaks and weekends. RMSE/MAE is about 1.44 to 1.58 for the models, versus 1.83 for seasonal naive on squares 5161 and 5259.

6.3 Computational cost

model	train_seconds_mean	train_seconds_min	train_seconds_max	inference_seconds_week	inference_ms_per_step	n_parameters	best_epoch
Persistence	0	0	0	0.065	0.065	0	-
Seasonal naive	0	0	0	0.066	0.065	0	-
SARIMA	1.162	0.743	1.666	0.115	0.114	3	-
LSTM	1005	266.703	1929	0.687	0.682	17217	27.000
TCN	644.485	193.046	1371	0.371	0.368	5601	26.333

Hardware details are in `results/hardware.json` (i5-6300U, 7.9 GB RAM, no GPU, two PyTorch threads).

These times are noisy. Free RAM fell to about 466 MB during the final runs, and the same LSTM setup cost about 10.5 seconds per epoch in tuning but about 88 seconds per epoch on square 5161 later. Only rough order-of-magnitude comparisons are trusted.

Even so, SARIMA fits in about one second and still loses to persistence. The TCN trains faster than the LSTM and uses fewer parameters (5,601 versus 17,217) while scoring better on average. Inference is under one millisecond per step for every model.

6.4 Failure analysis

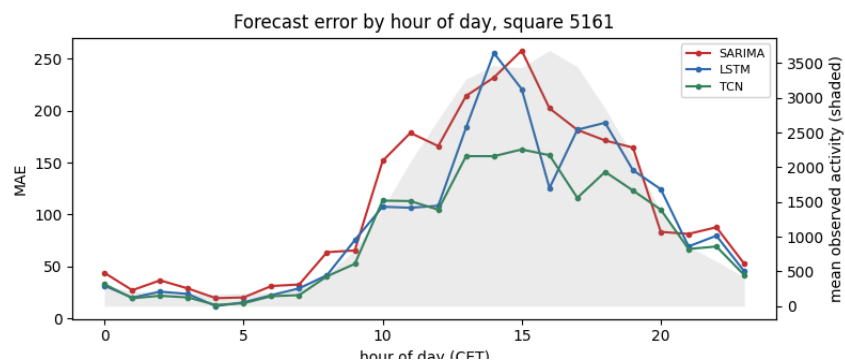


Figure 9. Square 5161: MAE by hour of day, with mean activity shaded.

At night all models look similar. Around the afternoon peak, SARIMA and LSTM rise to about 250 MAE while the TCN stays near 160. Most of the TCN gain comes from peak hours.

Worst six-hour windows (always a midday to evening peak):

Area	Worst window	SARIMA	LSTM	TCN
5161	Sat 21 Dec, 13:10 to 19:00	304.0 (2.81x)	201.0 (2.16x)	148.2 (1.89x)
5059	Tue 17 Dec, 12:20 to 18:10	158.6 (1.78x)	173.7 (2.58x)	164.8 (2.39x)
5259	Mon 16 Dec, 12:20 to 18:10	221.7 (2.71x)	131.0 (1.95x)	118.6 (1.83x)

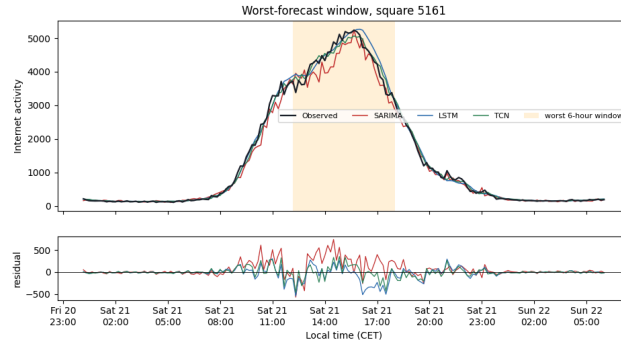


Figure 10. Square 5161 worst window: Saturday 21 December afternoon, after a 48% rise from Friday's peak.

Weekday peaks that week sit between about 3,057 and 3,877. Saturday reaches 5,238 and Sunday 5,496. This is not outside the training range: the training period includes a Saturday peak of 8,044. The hard part is the Friday to Saturday jump (3,530 to 5,238, +48%) with no day-of-week feature.

- SARIMA bias about -207 (under-predicts, locked to yesterday).
- LSTM bias about +112 (overshoots the descent).
- TCN bias about +15 (smallest and nearly unbiased).

6.5 Summary

	SARIMA	LSTM	TCN
Mean ratio to persistence	1.112	0.905	0.848
Spread across areas	0.088	0.179	0.007
Beats persistence	0/3	2/3	3/3
Parameters	3	17,217	5,601
Mean train time	1.2 s	1,005 s	644 s

The TCN is the best overall choice here: more accurate on average, more stable across areas, and cheaper than the LSTM. That fits prior work once horizon is taken into account [2], [6], [21], [29]. Spatio-temporal hybrids such as [28] answer a different question, because they use the full grid and neural baselines only.

7. Conclusion and Future Work

7.1 Findings

1. **Which baseline you use changes the story.** MASE against seasonal naive looks excellent for every model, but persistence is harder to beat, and SARIMA loses to it on all three areas.
2. **The TCN is the most reliable of the three.** It gains about 15% on persistence with almost no spread across areas. The LSTM is competitive on average but less stable.
3. **Receptive field did not matter at one step.** With matched epochs, depths from RF 61 to 509 were indistinguishable within noise. Daily seasonality still matters for SARIMA.
4. **The largest failure is a day-of-week change.** Errors concentrate at peaks. The worst case is the Friday to Saturday rise, not a Christmas peak beyond the training range.

7.2 Limitations

One seed per neural run; wall-clock stopping (including one truncated LSTM fit); noisy timing on a busy laptop; only three busy central areas; SARIMA without seasonal AR/MA terms; univariate inputs only; symmetric metrics [8].

7.3 Future work

1. Add day-of-week and holiday features.
2. Repeat key runs with several seeds on an unloaded machine.
3. Sweep horizons (1, 6, 36, 144 steps) [2].
4. Test quieter areas as well as busy ones.
5. Try spatial models [4], [15] if more channels are re-ingested.
6. Train with an asymmetric cost [8].

8. Academic Integrity

This is an individual submission. Documentation, textbooks, and the papers in Section 9 were used while learning the methods. Occasional help from a programming assistant was limited to clarifying language or library usage. It was not used in place of understanding the problem, designing the experiments, reading the results, or writing the argument of this report.

All numbers come from the code and experiments in the project repository [33]. I am responsible for the work under my name and can explain the data handling, model choices, methods, results, and conclusions.

9. References

- [1] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Scientific Data*, vol. 2, art. no. 150055, Oct. 2015, doi: 10.1038/sdata.2015.55.
- [2] C. Zhang and P. Patras, "Long-term mobile traffic forecasting using deep spatio-temporal neural networks," in *Proc. 19th ACM Int. Symp. Mobile Ad Hoc Networking and Computing (MobiHoc '18)*, Los Angeles, CA, USA, Jun. 2018, pp. 231–240, doi: 10.1145/3209582.3209606.
- [3] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, "Citywide cellular traffic prediction based on densely connected convolutional neural networks," *IEEE Communications Letters*, vol. 22, no. 8, pp. 1656–1659, Aug. 2018, doi: 10.1109/LCOMM.2018.2841832.
- [4] C. Zhang, H. Zhang, J. Qiao, D. Yuan, and M. Zhang, "Deep transfer learning for intelligent cellular traffic prediction based on cross-domain big data," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1389–1401, Jun. 2019, doi: 10.1109/JSAC.2019.2904363.
- [5] C.-W. Huang, C.-T. Chiang, and Q. Li, "A study of deep learning networks on mobile traffic forecasting," in *Proc. IEEE 28th Annu. Int. Symp. Personal, Indoor, and Mobile Radio Communications (PIMRC)*, Montreal, QC, Canada, Oct. 2017, pp. 1–6, doi: 10.1109/PIMRC.2017.8292737.
- [6] X. Wang, Z. Zhou, F. Xiao, K. Xing, Z. Yang, Y. Liu, and C. Peng, "Spatio-temporal analysis and prediction of cellular traffic in metropolis," *IEEE Transactions on Mobile Computing*, vol. 18, no. 9, pp. 2190–2202, Sep. 2019, doi: 10.1109/TMC.2018.2870135.
- [7] H. D. Trinh, L. Giupponi, and P. Dini, "Mobile traffic prediction from raw data using LSTM networks," in *Proc. IEEE 29th Annu. Int. Symp. Personal, Indoor and Mobile Radio Communications (PIMRC)*, Bologna, Italy, Sep. 2018, pp. 1827–1832, doi: 10.1109/PIMRC.2018.8581000.
- [8] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Pérez, "DeepCog: Cognitive network management in sliced 5G networks with deep learning," in *Proc. IEEE INFOCOM 2019*, Paris, France, Apr. 2019, pp. 280–288, doi: 10.1109/INFOCOM.2019.8737488.
- [9] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2224–2287, 3rd Quart. 2019, doi: 10.1109/COMST.2019.2904897.
- [10] Y. Shu, M. Yu, J. Liu, and O. W. W. Yang, "Wireless traffic modeling and prediction using seasonal ARIMA models," in *Proc. IEEE Int. Conf. Communications (ICC)*, May 2003, vol. 3, pp. 1675–1679, doi: 10.1109/ICC.2003.1203886.
- [11] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [13] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271 [cs.LG], Mar. 2018, doi: 10.48550/arXiv.1803.01271. (Technical report; not a peer-reviewed conference paper.)
- [14] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," arXiv:1609.03499 [cs.SD], Sep. 2016, doi: 10.48550/arXiv.1609.03499.
- [15] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, Montreal, QC, Canada, Dec. 2015. [Online]. Available: <https://arxiv.org/abs/1506.04214>
- [16] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, Oct.–Dec. 2006, doi: 10.1016/j.ijforecast.2006.03.001.
- [17] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.
- [18] X. Wang, K. Smith, and R. Hyndman, "Characteristic-based clustering for time series data," *Data Mining and Knowledge Discovery*, vol. 13, no. 3, pp. 335–364, Nov. 2006, doi: 10.1007/s10618-005-0039-x.
- [19] D. A. Dickey and W. A. Fuller, "Distribution of the estimators for autoregressive time series with a unit root," *Journal of the American Statistical Association*, vol. 74, no. 366, pp. 427–431, Jun. 1979, doi: 10.1080/01621459.1979.10482531.
- [20] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root?," *Journal of Econometrics*, vol. 54, no. 1–3, pp. 159–178, Oct.–Dec. 1992, doi: 10.1016/0304-4076(92)90104-Y.
- [21] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLoS ONE*, vol. 13, no. 3, art. no. e0194889, Mar. 2018, doi: 10.1371/journal.pone.0194889.
- [22] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, Jan.–Mar. 2020, doi: 10.1016/j.ijforecast.2019.04.014.
- [23] GSMA Intelligence, "Going green: Benchmarking the energy efficiency of mobile networks (second edition)," GSMA Intelligence, London, U.K., Feb. 2023. (Industry report; not peer-reviewed.)
- [24] L. M. P. Larsen, H. L. Christiansen, S. Ruepp, and M. S. Berger, "Toward greener 5G and beyond radio access networks: A survey," *IEEE Open Journal of the Communications Society*, vol. 4, pp. 768–797, 2023, doi: 10.1109/OJCOMS.2023.3257889.
- [25] J. Wu, Y. Zhang, M. Zukerman, and E. K.-N. Yung, "Energy-efficient base-stations sleep-mode techniques in green cellular networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 2, pp. 803–826, 2nd Quart. 2015, doi: 10.1109/COMST.2015.2403395.
- [26] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.
- [27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, May 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [28] Z. Zhang, S. Gong, Z. Liu, and D. Chen, "A novel hybrid framework based on temporal convolution network and transformer for network traffic prediction," *PLOS ONE*, vol. 18, no. 9, art. no. e0288935, Sep. 2023, doi: 10.1371/journal.pone.0288935.

[29] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers effective for time series forecasting?," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11121–11128, doi: 10.1609/aaai.v37i9.26317.

Software

[30] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[31] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 92–96, doi: 10.25080/Majora-92bf1922-011.

[32] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Vancouver, BC, Canada, Dec. 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>

Project artefacts

[33] Source code repository: <https://github.com/Emmanuel-NS/Time-series-forecasting-FA>

[34] Video presentation: https://youtu.be/EzIG6sXS4_c